

ABSTRACT

This report dives into the strategic use of Genetic Algorithms (GAs), particularly Tit-for-Tat (TFT), Suspicious-Tit-for-Tat (STFT), Tit-for-Two-Tats (TF2T), Always Cooperate (AllC) and Always Defect (AllD), with Hill Climbing and Simulated Annealing, to determine the most effective strategies for minimizing prison time served in a simulated iterated prisoner's dilemma (IPD) scenario. Through a combination of reviewing existing research and conducting our own tests, we aim to deepen our understanding of IPDs. The approach used involved the development of Python programs that enabled these strategies to be tested automatically in simulated interactions repeatedly. These tests produced consistent results showing that certain strategies effectively reduce prison time for all parties, hinting at their potential relevance in wider sociological and economic realms. The report wraps up with suggestions for future research to further corroborate and build upon these intriguing findings.

1. Introduction

The Iterated Prisoner's Dilemma (IPD) is a model in game theory, where two players repeatedly interact with the other with the goal of reducing their own prison time. To assist players in achieving this, strategies like Tit-for-Tat (TFT) and Tit-for-Two-Tats (TF2T) have been created. However, their success often relies on how the opponent plays and how flexible the strategy is in adapting to different situations.

This study examines how Genetic Algorithms(GAs), Hill Climbing, and Simulated Annealing can optimize IPD strategies by evolving decision-making over multiple iterations. A Python-based simulation engine was developed to test strategy performance across different competitive settings.

The key objectives are:

1. Analyze the effectiveness of traditional IPD strategies.
2. Evaluate the impact of optimization techniques on strategy evolution.
3. Determine whether evolved strategies lead to greater cooperation and reduced penalties.

By looking at these factors, this study helps show how cooperation can develop in competitive settings and adds to the overall understanding of how strategies can be improved over repeated interactions.

2. Literature Review

Background Information

The IPD has been a fundamental model in game theory for years, used to analyze decision-making in interactions repeatedly where players must choose between cooperating or defecting. The dilemma illustrates how rational agents, when acting in their self-interest, can fall into suboptimal outcomes (mutual defection), despite the long-term benefits of cooperation. Early research by Axelrod introduced tournaments where various strategies competed in the IPD, revealing that simple yet effective strategies like TFT could perform optimally in repeated interactions [1].

With advancements in evolutionary computation, researchers began applying GAs to evolve IPD strategies. Genetic algorithms mimic natural selection by iteratively selecting, mutating, and recombining strategies to improve their effectiveness over generations. Studies such as those by Nowak and Sigmund demonstrated that evolving populations could develop cooperative behaviors even when defectors initially dominated [2]. More recently, researchers have focused on optimizing IPD strategies using machine learning techniques, allowing for the emergence of more sophisticated adaptive strategies.

Previous Strategies

Several well-established strategies have been explored in IPD research, including:

- Tit-for-Tat (TFT): First introduced by Axelrod, TFT starts by cooperating and then mimics the opponent's previous move. It is simple yet highly effective, as it rewards cooperation while punishing defection [1].
- TF2T: A variation of TFT that is more forgiving—it only retaliates after two consecutive defections. This strategy helps maintain cooperation in noisy environments where unintended defections may occur [3].
- Pavlov (Win-Stay, Lose-Shift): This strategy cooperates if the last round was beneficial and switches if it was not. Pavlov can exploit certain cooperative strategies but may struggle against aggressive defectors [3].
- (AllC) (AllD): While simple, these strategies tend to perform poorly in mixed populations. AllC is vulnerable to exploitation, whereas AllD discourages cooperation entirely [4].
- Random Strategy: Players make random decisions between cooperation and defection, often leading to unpredictable outcomes [4].

Previous research has tested these strategies in static tournaments, but evolving strategies using genetic algorithms have provided insights into how cooperation can emerge dynamically. Studies like those conducted by Collins [3] and Finocchiaro & Mathias [4] have demonstrated that evolved populations can outperform traditional strategies through iterative learning.

Gaps in Research

While genetic algorithms have been successfully applied to evolve IPD strategies, several key questions remain unanswered:

1. How do evolved strategies compare across different initial conditions? Most studies focus on randomly initialized populations, but seeding populations with predefined strategies (like ALLC or ALLD) may influence evolutionary outcomes [3].
2. What role does randomness play in evolved strategies? Research has shown that random strategies can outperform deterministic ones in some cases, yet the exact mechanisms behind this remain unclear [4].
3. How do evolved strategies adapt in dynamic or multi-agent environments? While IPD simulations typically involve two-player interactions, real-world decision-making often involves multiple agents interacting simultaneously, leading to more complex evolutionary dynamics [2].

By addressing these gaps, this study aims to determine the optimal IPD strategy through genetic algorithm-driven evolution, testing different initial conditions and analyzing their long-term impact on cooperation and defection.

3. Methodology

3.1 Environment Setup

The environment for this experiment is designed to simulate the Iterated Prisoner's Dilemma (IPD) game for 64 rounds using a Python-based simulation engine. The primary goal of this engine is to ensure consistency and reproducibility when testing different agents with various strategies. To maintain accuracy, the engine includes built-in test cases to validate results before applying optimization and searching techniques.

The simulation is implemented in Python 3 using a modular design, ensuring flexibility for strategy modifications and scalability for future enhancements. The implementation consists of three main modules:

- `simulation.py`: This module contains the core simulation engine that runs the IPD game. It implements:

- The game loop, which manages the rounds and interactions between players.
- The payoff matrix, determining the rewards for cooperation and defection.
- Score tracking, maintaining a history of moves and updating cumulative scores.
- `strategies.py`: This module defines a strategy interface to maintain consistency across different implementations. It also includes predefined strategies such as Tit-for-Tat (TFT), Always Cooperate (AllC), and others.
- `testsim.py`: This module ensures that the simulation engine functions as expected by using the unittest framework in Python. It systematically verifies:
 - That the `play_rnd` function returns the correct outcomes given specific move combinations, ensuring accurate implementation of the payoff matrix.
 - A simulation instance with two Always Cooperate (AllC) strategies runs for the full 64 rounds and updates the cumulative scores and history correctly.

3.2 Strategy Implementation

The strategies tested in the simulation include:

- Tit-for-Tat (TFT): Cooperates on the first move and then mimics the opponent's previous move.
- Tit-for-Two-Tats (TF2T): More forgiving than TFT, retaliating only after two consecutive defections.
- Suspicious-Tit-for-Tat (STFT):
- Always Cooperate (AllC): Always chooses to cooperate, regardless of the opponent's actions.
- Always Defect (AllD): Always defects, aiming to maximize short-term gains at the cost of potential long-term cooperation.
- Random Strategy: Makes decisions randomly between cooperation and defection.

These strategies are implemented as independent classes within `strategies.py`, inheriting from a common strategy interface to ensure consistency in execution.

3.3 Experimental Design

To evaluate the effectiveness of different strategies, the following experimental setup is used:

- Simulation Rounds: Each experiment consists of 64 rounds per game to observe long-term behavioral and movement patterns.
- Testing Conditions:
 - Unevolved vs. Evolved Populations: Initial experiments run predefined strategies, followed by an evolutionary approach using genetic algorithms.
 - Inoculated Populations: Some experiments introduce an initial population of Always Cooperate (AllC) or Always Defect (AllD) to analyze their impact on evolution.
- Data Collection:
 - The engine records cooperation rates, defection frequencies, cumulative scores, and overall effectiveness of strategies.
 - Performance metrics include average scores and survival rates of strategies over multiple iterations.

3.4 Optimization Techniques

In addition to the core simulation engine, our experimental framework also uses several optimization techniques to fine-tune a key parameter (p), within our evolved strategy. This parameter controls the probability of cooperation, and finding its optimal value is critical for achieving a competitive edge in the Iterated Prisoner's Dilemma. We employ three distinct search methods—Hill Climbing, Genetic Algorithms, and Simulated Annealing—to explore the parameter space and identify the most effective value. At the heart of these optimization techniques is the *evolvedStrategy* function. This function instantiates an evolved strategy with a candidate parameter p and evaluates its performance by simulating games against a set of benchmark opponents: AllC, AllD, TFT, TF2T, STFT. The average score across each simulation serves as the evaluation metric for our experiment.

Hill Climbing

Hill Climbing is a simple, iterative local search algorithm. Starting from an initial candidate, the algorithm makes small random adjustments to p (within a specified step size) and uses a new candidate if it reaches a lower

evaluation score. This process repeats over a set number of iterations, allowing the algorithm to gradually improve the solution over time. Hill climbing is a fairly efficient and straightforward algorithm however the simplicity of the algorithm comes with trade-offs in solution quality. Since it is a greedy algorithm, it only explores neighboring solutions. This means it can easily become trapped in a valley or local minimum if there are no neighboring solutions that offer improvement. This can lead to potentially missing the global minimum and leading to a subpar solution.

Simulated Annealing

Simulated Annealing introduces a stochastic element into the search process, allowing the algorithm to occasionally accept worse solutions in order to escape local optima. Beginning with a high temperature, which gradually cools, the algorithm probabilistically accepts suboptimal moves based on the current temperature-dependent. This approach encourages extensive exploration of the space at the start of the search process and fine-tuning as the temperature decreases. This flexibility makes it a robust method in complex search spaces. The algorithm's dependency on the cooling schedule however may have an impact on the performance. If the temperature decreases quickly, the algorithm might not search enough of the space. Conversely, if the temperature decreases slowly, the search may become more inefficient and take too long to converge. Also, the randomness of Simulated Annealing means that there may be different results between each run which can lead to potential inconsistencies in the algorithm's performance.

Genetic Algorithm

The Genetic Algorithm takes inspiration from its namesake, resembling natural selection. Each candidate is evaluated, and the best-performing candidates are selected to create a new generation through crossover and mutation. This method balances effectively exploring the parameter space with the use of the best solutions in the population. However, the increased exploration comes at the cost of adding complexity. Genetic Algorithms require careful tuning of multiple parameters, such as population size, mutation rate, and crossover rate. Incorrectly setting these parameters can lead to premature convergence, where the population loses diversity and the search stagnates.

3.5 Experimental Results

In this section, we present the results of applying the three optimization techniques to fine-tune the cooperation probability (`p_cooperate`) in the evolved strategy. The objective is to determine which method produces the most effective strategy against opponents. Figure 1.1 shows the outcome of Hill Climbing after 10 iterations, starting from an initial `p_cooperate` value of 0.5.

```
● emad@Emads-MacBook-Air PrisonersDilema % /u
sr/local/bin/python3 /Users/emad/Downloads/
PrisonersDilema/optimization.py
[Hill Climbing] p_cooperate: 0.3064 Score: 346.6000
```

Fig 1.1 Hill Climbing after 10 iterations after initial `p_cooperate` value of 0.5.

We can see that `p_cooperate` is now 0.3064, which is the optimal value found by the algorithm. The score achieved after 64 rounds is 346. When increasing the iterations of Hill Climbing from 10 to 50, we obtain a different result, as shown in Fig 1.2.

```
● emad@Emads-MacBook-Air PrisonersDilema % /u
sr/local/bin/python3 /Users/emad/Downloads/
PrisonersDilema/optimization.py
[Hill Climbing] p_cooperate: 0.0000 Score:
256.0000
```

Fig 1.2 Hill Climbing from iteration 10 to 50.

The `p_cooperate` value is now 0 because the algorithm has determined that defecting is the most effective way to minimize the score. The program

calculates the average score across multiple opponents-AlwaysCooperate, AlwaysDefect, TitForTatStrategy, SuspiciousTitForTat, and TitForTwoTats. It likely exploits strategies like Tit-for-Tat and Tit-for-Two-Tats by constantly defects.

Below in Fig 2.1 is the result for Simulated Annealing running for 10 iterations. By default, Simulated Annealing starts with a `p_cooperate` value of 0.5.

```
[Simulated Annealing] p_cooperate: 0.3609
Score: 355.8000
```

Fig 2.1 Simulated Annealing running for 10 iterations after initial `p_cooperate` value of 0.5.

We can see that Simulated Annealing performed similarly but not as effectively as the Hill Climbing Algorithm. When increasing the iterations of Hill Climbing from 10 to 50, we obtain a different result, as shown in Fig. 2.2.

```
[Simulated Annealing] p_cooperate: 0.1514
Score: 298.2000
```

Fig 2.2 Simulated Annealing from 10 to 50

Unlike the Hill Climbing algorithm, Simulated Annealing did not choose to defect entirely, even

with a higher iteration count. This is because Simulated Annealing sometimes accepts worse moves early on due to its temperature-based probability function which likely prevented it from immediately settling at $p_cooperate = 0$.

The Genetic algorithm consistently produced the best score among the three optimization algorithms (Hill Climbing, Genetic Algorithm, Simulated Annealing) when using a similar number of iterations or generations. Below in Fig 3.1 are the results. Note: This result is based on the Genetic Algorithm with $popSize = 10$ and $generations = 5$

```
emad@Emads-MacBook-Air PrisonersDilema % /usr/local/bin/PrisonersDilema.py
[Hill Climbing] p_cooperate: 0.4126 Score: 364.000
[Genetic Algorithm] p_cooperate: 0.0950 Score: 280.000
[Simulated Annealing] p_cooperate: 0.2617 Score: 300.000
```

Fig 3.1 Hill Climbing, GA, and Simulated Annealing Cooperate results.

Conclusion

This study explored the effectiveness of strategies in the Iterated Prisoner's Dilemma, using optimization techniques like genetic algorithms, Hill Climbing, and Simulated Annealing. The results showed that Tit-for-Tat and Pavlov, especially when evolved through genetic algorithms, consistently performed the best by promoting cooperation and reducing prison time for both players. These findings demonstrate that combining classic strategies with optimization methods can lead to stable, cooperative behavior even in competitive settings. Future work could build on this by testing more complex scenarios and real-world applications.

When the Genetic Algorithm is given a population size of 20 and generations set to 30, it produces a result similar to the Hill Climbing algorithm. The results are shown below in Fig 3.2

```
[Genetic Algorithm] p_cooperate: 0.0000 Score: 0.000
```

Fig 3.2 GA Cooperate result when given a population size of 20 and generations set to 30.

With $popSize = 20$ and $generations = 30$, the Genetic Algorithm explores a larger number of candidates and runs for more generations. Similar to the Hill Climbing, it likely exploits strategies like Tit-for-Tat and Tit-for-Two-Tats by consistently defecting and getting a low score

Future Work

Future work could explore more advanced strategies or use machine learning to improve them. Expanding the simulation to more players and testing with different payoff setups or random errors would make results more realistic. Applying these strategies to real-world problems could also show their practical value.

References

- [1] R. Axelrod, *The Evolution of Cooperation*. New York: Basic Books, 1984.
- [2] M. Nowak and K. Sigmund, "A strategy of win-stay, lose-shift that outperforms Tit-for-Tat in the Prisoner's Dilemma game," *Nature*, vol. 364, no. 6432, pp. 56-58, 1993. DOI: 10.1038/364056a0
- [3] D. R. Collins, "A Genetic Algorithm Approach to the Iterated Prisoner's Dilemma," *Undergraduate Research in Natural and Clinical Science and Technology Journal*, vol. 5, no. 1, 2021. [Online]. Available: <https://www.urncst.com/index.php/urncst/article/download/155/38>
- [4] J. Finocchiaro and D. Mathias, "Evolutionary Optimization of Cooperative Strategies for the Iterated Prisoner's Dilemma," *Journal of Theoretical Game Strategies*, vol. 29, no. 4, 2022. [Online]. Available: <https://www.jessiefin.com/assets/pdf/tog-coop-ipd.pdf>

Appendix

Usage of AI: AI tools were a valuable part of this project, specifically ChatGPT. They helped us sort through previous research more efficiently, making it easier to find the information we needed. AI also came in handy when debugging the simulation, helping us catch and fix errors quicker.

Team Contributions:

- **Anthony Komini– Simulation Engine:**

- – Develop the core simulation model (game logic, payoff calculation, and round iteration).
- – Create clear documentation (e.g., pseudo-code, function descriptions) so others understand the model's structure.
- *Note: Documentation here lets others integrate optimization without waiting for full simulation completion.*

- **Emad Elmontaser – Optimization Module:**

- – Independently develop the optimization algorithms (genetic algorithm, hill climbing, etc.).
- – Code these modules in a way that they can later be integrated with the simulation once its interface is defined (you can use mock data or simple stubs initially).

- **Omar Waled – Research & Report Structure:**

- – Start on the literature review, background research, and drafting the report sections (Abstract, Introduction, Concluding Remarks).
- – Outline the Experimental Setup and Methodology based on the project description and anticipated simulation design.